Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Master Thesis – Structure Proposal C

Template C: Design Science – Building a Fair Comparison

by

Author's Name

7th October, 2026

University of Kaiserslautern-Landau
Department of Computer Science
Distributed Systems Lab

Examiner: Prof. Dr.-Ing. Jens B. Schmitt
M. Sc. Supervisor Name

About This Proposal

Organising idea. The thesis follows the design science research process: problem, objectives, design and development, demonstration, evaluation [1]. It presents *two* artefacts. Artefact I is WADM, the deception layer. Artefact II is the method that makes four different edges comparable: behavioural and tuning levelling, plus a paired benchmark with validity gates. The flaws found and fixed along the way become content, in the spirit of benchmarking-flaw studies [2].

Choose it if you want the levelling and methodology work, a large share of the project's effort (`EDGE_LEVELING.md`, `BENCHMARK_METHODOLOGY.md`), credited as a contribution in its own right, and your examiners care most about measurement rigour.

Trade-off. This is the longest of the three structures, and it risks reading like a chronicle. Keep the history to the flaws that changed a conclusion, each with before-and-after evidence.

Ch.	Title	approx. pages
1	Introduction	5
2	Foundations	9
3	Related Work	6
4	Problem Analysis and Objectives	7
5	Artefact I: The Deception Layer	12
6	Artefact II: A Fair Cross-Edge Evaluation Method	14
7	Demonstration	6
8	Evaluation	18
9	Discussion	7
10	Conclusion and Future Work	3
A–D	Appendices	as needed

Page counts are a rough budget, not a department rule.

How to read the rest. The table of contents is the proposed structure. Under each heading is a short note on what to write there, and a *Material* line pointing to the place in the repository that already holds it (`results/` and `plots/` are short for `benchmarks/results/` and `benchmarks/results/plots/`).

Front matter (unchanged from the department template)

- **Title page** and **Eigenständigkeitserklärung**, as generated by `rptuma.cls`. The declaration requires disclosing generative-AI use beyond grammar and style.
- **Abstract** (English): the problem (deception at the web edge has an unknown cost), the approach (one mechanism on four edges, levelled to identical behaviour), the method (paired benchmark on three measurement planes) and the headline numbers.
- **Zusammenfassung**: the same abstract in German.
- **Contents**, **List of Figures**, **List of Tables**.
- **Glossary** (optional, the class's `glossar` environment): WADM, honeytoken, edge, tier, VU, iteration, SLO, cgroup, LuaJIT, PUC Lua, proxy-wasm.

In the department template's own example, websites are cited through an archived copy (`web.archive.org`); do the same for any web source in the list at the end.

Contents

1 Introduction **1**
- 1.1 Problem and Motivation . . . 1
- 1.2 Research Approach . . . 1
- 1.3 Contributions . . . 1
- 1.4 Outline . . . 1

2 Foundations **2**
- 2.1 Deception and Honeytokens . . . 2
- 2.2 Edge Proxies and Extension Runtimes . . . 2
- 2.3 Performance Evaluation Foundations . . . 2

3 Related Work **3**
- 3.1 Deception at the Application Layer . . . 3
- 3.2 Measuring Proxy Overhead . . . 3
- 3.3 Benchmarking Flaws in Systems Security . . . 3

4 Problem Analysis and Objectives **4**
- 4.1 Scenario and Stakeholders . . . 4
- 4.2 Threat Model . . . 4
- 4.3 Functional Objectives . . . 4
- 4.4 Non-Functional Objectives . . . 4
- 4.5 Comparability Objectives . . . 4

5 Artefact I: The Deception Layer **5**
- 5.1 Architecture and Configuration . . . 5
- 5.2 Honeytoken Kinds . . . 5
  - 5.2.1 HTML Comments and HTTP Headers . . . 5
  - 5.2.2 Cookies and Form Fields . . . 5
  - 5.2.3 Decoy Paths . . . 5
- 5.3 The Fake SQL-Injection Trap . . . 5
- 5.4 Realisation on Four Edges . . . 6
  - 5.4.1 OpenResty . . . 6
  - 5.4.2 Envoy with Lua . . . 6
  - 5.4.3 Envoy with WebAssembly . . . 6
  - 5.4.4 Apache httpd with mod_lua . . . 6

6 Artefact II: A Fair Cross-Edge Evaluation Method **7**
6.1 Levelling Behaviour . . . 7
6.2 Levelling Product Defaults . . . 7
6.3 Baselines and Tiers . . . 7
6.4 Measurement Planes . . . 7
6.5 From a Flawed Harness to a Paired One . . . 8
6.5.1 Flaws Found . . . 8
6.5.2 Fixes . . . 8
6.6 Validity Gates and Negative Controls . . . 8

7 Demonstration **9**
7.1 Behavioural Parity . . . 9
7.2 Functional Behaviour across Response Shapes . . . 9
7.3 An Attack Walk-Through . . . 9

8 Evaluation **10**
8.1 Setup and Protocol . . . 10
8.2 Latency . . . 10
8.3 CPU, Cost Attribution and the Probe Effect . . . 10
8.4 Capacity . . . 10
8.5 Mechanisms, Page Size and the SQLi Trap . . . 10
8.6 Stability: Apache mod_lua . . . 11
8.7 Evaluating the Method Itself . . . 11

9 Discussion **12**
9.1 Meeting the Objectives . . . 12
9.2 Lessons for Cross-Platform Benchmarks . . . 12
9.3 Threats to Validity and Limitations . . . 12

10 Conclusion and Future Work **13**
10.1 Summary . . . 13
10.2 Future Work . . . 13

A Configuration Reference **14**

B Equivalence Invariants Checklist **15**

C Harness History and Archived Results **16**

D Reproduction Guide **17**

1 Introduction

1.1 Problem and Motivation

Deception at the edge needs no change to the application, but its cost is unknown, and comparing it across edges is harder than it looks.

1.2 Research Approach

Design science with two artefacts, each built, demonstrated and evaluated in iterations.

1.3 Contributions

WADM on four edges; the levelling rules and parity check; the paired three-plane harness; the measured costs; a catalogue of the comparison pitfalls found along the way.

1.4 Outline

One line per chapter, mapped to the design science steps.

2 Foundations

2.1 Deception and Honeytokens

Honeypots, honeytokens, and why interaction with bait is a high-signal alert [3, 4]

2.2 Edge Proxies and Extension Runtimes

Request phases and buffering; LuaJIT, PUC Lua and WebAssembly.

2.3 Performance Evaluation Foundations

Latency and its percentiles, cgroup CPU accounting, open- and closed-loop load, pairing, replication, crossover designs [5, 6].

3 Related Work

3.1 Deception at the Application Layer

Decoys, honeywords and deceptive web artefacts, and deploying them without developer involvement [7, 8, 9, 10]

3.2 Measuring Proxy Overhead

How sidecar and proxy costs have been measured, and which of their choices this harness adopts [11].

3.3 Benchmarking Flaws in Systems Security

[2, 12]. Which of the documented flaws this project ran into itself.

4 Problem Analysis and Objectives

Design science step 2: the objectives against which the artefacts are later judged.

4.1 Scenario and Stakeholders

An operator protects a web application whose code it cannot change.

4.2 Threat Model

A reconnaissance client reads HTML, replays values, tampers with cookies and form fields, and probes for SQL injection.

4.3 Functional Objectives

F1 plant bait; F2 detect interaction with it; F3 strip the trigger before it reaches the origin; F4 raise one alert per detection; F5 driven only by configuration.

4.4 Non-Functional Objectives

N1 low added latency; N2 low CPU cost; N3 no broken responses; N4 portable to four edges.

4.5 Comparability Objectives

C1 identical behaviour; C2 identical timed regions; C3 levelled product defaults; C4 a baseline that differs only by WADM.

5 Artefact I: The Deception Layer

5.1 Architecture and Configuration

Edge between client and static origin; one shared `config.json`.

Material: `CONTEXT.md`, `config.json`

5.2 Honeytoken Kinds

5.2.1 HTML Comments and HTTP Headers

Planted secrets, detected when the keyword is replayed.

5.2.2 Cookies and Form Fields

Planted values, detected when tampered with.

5.2.3 Decoy Paths

An advertised link to a trap path; any request to it matches.

5.3 The Fake SQL-Injection Trap

A detection-only feature: the edge answers itself with a fake MySQL error.

5.4 Realisation on Four Edges

5.4.1 OpenResty

The reference edge: one precompiled table, four phase hooks, a shared dictionary for IPs.

Material: `nginx/README.md`

5.4.2 Envoy with Lua

One filter script; the request path is handed to the response phase.

Material: `envoy_scripts/README.md`

5.4.3 Envoy with WebAssembly

Rust proxy-wasm; fewer host calls; the trap still contacts the origin (a host constraint).

Material: `wasm-filter/README.md`

5.4.4 Apache httpd with mod_lua

A shared core for three hooks; per-thread Lua states; PUC Lua rather than LuaJIT.

Material: `apache_scripts/README.md`

6 Artefact II: A Fair Cross-Edge Evaluation Method

The core methodological contribution.

6.1 Levelling Behaviour

Differences found: Apache never stripped the keyword (Lua pattern characters in the keyword), four different detection mechanisms, attacker IPs recorded as `unknown` on Envoy, a log line on every request, PCRE against plain find. The canonical mechanism that replaced them.

Material: `docs/THESIS_NOTES.md` (2026-09-18), `docs/EDGE_LEVELING.md`

6.2 Levelling Product Defaults

Upstream keep-alive (about 0.5 ms per request on OpenResty), access logs, `LuaCodeCache`. The rule this establishes: level defaults before comparing.

Material: `docs/THESIS_NOTES.md` (2026-09-17)

6.3 Baselines and Tiers

Bare (WADM deleted, nothing added), silent (as deployed), instrumented (timers on).

6.4 Measurement Planes

Timers, end-to-end latency, cgroup CPU, and why no single plane survives a noisy VM.

6.5 From a Flawed Harness to a Paired One

6.5.1 Flaws Found

Request position read as WADM phase, a mean dominated by its tail, unpaired tiers, unequal offered load, environment variables silently overriding the scenario, log writes inside the measured path, the load generator competing with the edge.

6.5.2 Fixes

Pairing, balanced tier and probe orders, medians and trimmed means, separate tiers for instrumentation.

Material: `docs/BENCHMARK_METHODOLOGY.md` §1, §7

6.6 Validity Gates and Negative Controls

Gates scale with the damage rather than firing on any failure; origin CPU is the negative control.

Material: `benchmarks/wadm_timings.py`

7 Demonstration

Design science step 4: the artefacts work as intended.

7.1 Behavioural Parity

`parity_check.py`: identical responses, alerts and origin requests on all four edges.

Material: `benchmarks/parity_check.py`

7.2 Functional Behaviour across Response Shapes

Thirteen response types, bare against WADM: preserved, injected, changed or broken.

Material: `results/functional_matrix.md`

7.3 An Attack Walk-Through

Read the HTML comment, replay its keyword, tamper with a cookie, try SQL injection; the alerts that result on each edge.

8 Evaluation

Design science step 5: Artefact I judged against N1–N3, Artefact II against C1–C4.

8.1 Setup and Protocol

Testbed, workload, rates, replicates, rejected cells.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4, §7

8.2 Latency

+109 to +218 µs per `GET` at the median.

Material: `plots/paired_latency_added.png`

8.3 CPU, Cost Attribution and the Probe Effect

+33 to +200 µs of CPU per request; the timers enclose 5–22% of it.

Material: `plots/cost_attribution.png`

8.4 Capacity

Capacity kept with silent WADM, and the resource that bound it.

Material: `results/capacity_summary.md`

8.5 Mechanisms, Page Size and the SQLi Trap

The `GET` path against the trap, linear growth with body size, the 2×2.

Material: `results/mechanisms.md`, `results/sqli.md`

8.6 Stability: Apache mod_lua

Segfaults driven by cumulative requests, and the `LuaScope conn` trade-off.

Material: `plots/apache_remediation.png`

8.7 Evaluating the Method Itself

How much levelling and pairing changed the numbers. WASM's 500-VU p90 fell from 382 to 65 ms once behaviour was levelled. The old harness could not resolve the effect it claimed to measure, while the paired harness resolves every `GET` cell. The two-tier figures had overstated WADM's cost by the instrumentation's own cost.

Material: `docs/BENCHMARK_METHODOLOGY.md` intro and §8.2

9 Discussion

9.1 Meeting the Objectives

A table mapping each of F1–F5, N1–N4 and C1–C4 to its evidence, with a verdict.

9.2 Lessons for Cross-Platform Benchmarks

Level the defaults, pair the tiers, gate on damage, keep a negative control.

9.3 Threats to Validity and Limitations

Single VM host, synthetic traffic, POST-body inspection on only two edges, no TLS.

10 Conclusion and Future Work

10.1 Summary

What each artefact achieved, against the objectives.

10.2 Future Work

Complete the fourth edge's POST inspection, injection into compressed bodies, a field study, and reusing Artefact II for other edge features.

A Configuration Reference

Every `config.json` field per honeytoken kind, and the SQLi trap's fields.

Material: `CONTEXT.md`

B Equivalence Invariants Checklist

Each invariant, how it is checked, and the residual differences left on purpose.

Material: `benchmarks/README.md` (“Level-playing-field invariants”)

C Harness History and Archived Results

Each archived result set and why it was superseded.

Material: `benchmarks/results/archive/`

D Reproduction Guide

Commands and machine configuration.

Material: `docs/BENCHMARK_METHODOLOGY.md` §3–4

Sources Used for This Proposal

[1] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[2] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[3] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.

[4] L. Spitzner, "Honeytokens: The other honeypot." SecurityFocus, 2003.

[5] R. Jain, *The Art of Computer Systems Performance Analysis*. Wiley, 1991.

[6] E. J. Williams, "Experimental designs balanced for the estimation of residual effects of treatments," *Australian Journal of Scientific Research, Series A*, vol. 2, no. 2, pp. 149–168, 1949.

[7] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, "Baiting inside attackers using decoy documents," in *Security and Privacy in Communication Networks (SecureComm)*, pp. 51–70, 2009.

[8] A. Juels and R. L. Rivest, "Honeywords: Making password-cracking detectable," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pp. 145–160, 2013.

[9] N. Virvilis, B. Vanautgaerden, and O. S. Serrano, "Changing the game: The art of deceiving sophisticated attackers," in *6th International Conference on Cyber Conflict (CyCon)*, pp. 87–97, 2014.

[10] M. Kahlhofer and S. Rass, "Application layer cyber deception without developer interaction," in *3rd Workshop on Active Defense and Deception (ADnD), co-located with IEEE EuroS&P*, 2024. arXiv:2405.12852.

[11] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting service mesh overheads." arXiv:2207.00592, 2022.

[12] G. Heiser, "Systems benchmarking crimes." `https://gernot-heiser.org/benchmarking-crimes.html`. Accessed: 07.10.2026.